24

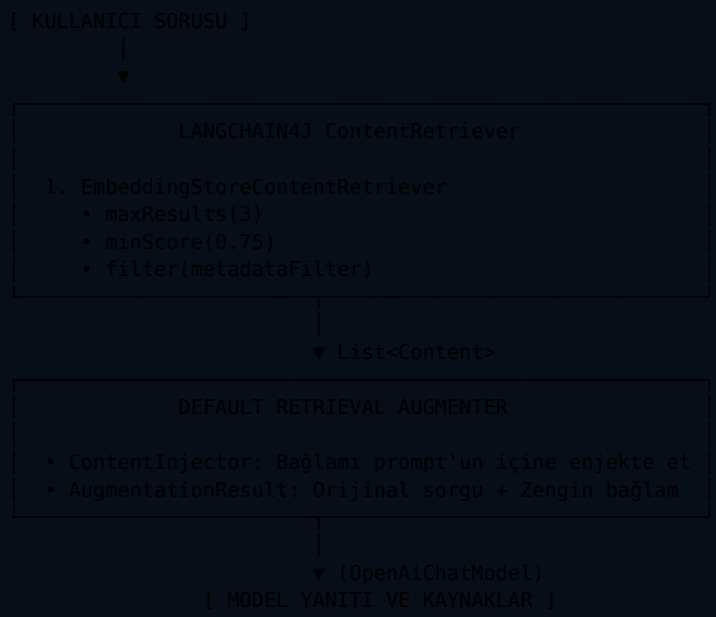
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

LangChain4j ContentRetriever ve AugmentationResult Mimarisi

LangChain4j ContentRetriever, Augmenter & Query Routing Architecture

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dedektifin olay yerinde delil arayan özel bir yardımcısı olduğunu düşünün (ContentRetriever). Dedektif 'Bana saat 22:00 civarındaki şüphelileri bul' dediğinde, yardımcısı arşiv odasına koşar, sadece o saatteki 3 güvenlik kamerası kaydını getirir (Content). Ardından bu delilleri dedektifin masasına koyup raporun üstüne iğneler (Augmenter). LangChain4j bu modüler dedektif yapısını sunar: Delil getirme mantığını modelden tamamen ayırabilirsiniz!

🇬🇧 ENGLISH (ELI5)

Imagine a detective who employs a specialized forensic archivist (ContentRetriever). When the detective queries: 'Find all suspects spotted near the vault after 10 PM', the archivist retrieves only the three matching surveillance logs (Content). The archivist pins these exhibits directly onto the case folder (RetrievalAugmentor). LangChain4j provides this modular architecture: the retrieval logic is decoupled from generation, enabling flexible routing, filtering, and multi-source aggregation!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LangChain4j RAG Genişletme Mimarisi: 1) `ContentRetriever`: Herhangi bir kaynaktan (Vektör veritabanı, Web araması, SQL, REST API) `Content` listesi çeken temel arayüzdür. 2) `EmbeddingStoreContentRetriever`: Vektör deposundan filtreli ve skor eşikli arama yapar. 3) `RetrievalAugmentor`: `QueryTransformer` (Sorguyu zenginleştirme), `ContentRetriever` (Veri çekme), `ContentAggregator` (Farklı kaynaklardan gelenleri birleştirme/sıralama) ve `ContentInjector` (Prompt'a enjekte etme) adımlarını yönetir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

LangChain4j Modular Retrieval Architecture: 1) `ContentRetriever`: Core SPI returning a `List\<Content>` given a `Query`. Extensible to hybrid vector stores, elastic indexes, or external REST endpoints. 2) `RetrievalAugmentor`: The composite orchestrator driving the RAG lifecycle: Query Routing, Query Transformation, Multi-Source Retrieval, Reciprocal Rank Fusion, and Prompt Injection. 3) Citation Support: By retaining segment source metadata, the augmenter enables verifiable reference tracking in enterprise compliance apps.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: ContentRetriever ile Deklaratif RAG Servisi
import dev.langchain4j.data.segment.TextSegment;
import dev.langchain4j.model.openai.OpenAiChatModel;
import dev.langchain4j.rag.content.retriever.EmbeddingStoreContentRetriever;
import dev.langchain4j.service.AiServices;
import dev.langchain4j.store.embedding.inmemory.InMemoryEmbeddingStore;

public class LangChain4jRagModule {
    public interface EnterpriseAssistant {
        String answer(String userQuery);
    }

    public static EnterpriseAssistant buildAssistant(InMemoryEmbeddingStore<TextSegment> store, String
apiKey) {
        var chatModel = OpenAiChatModel.builder()
            .apiKey(apiKey)
            .modelName("gpt-4o-mini")
            .temperature(0.0)
            .build();

        // Vektör deposundan içerik getiren retriever
        var contentRetriever = EmbeddingStoreContentRetriever.builder()
            .embeddingStore(store)
            .maxResults(3)
            .minScore(0.70)
            .build();

        return AiServices.builder(EnterpriseAssistant.class)
            .chatLanguageModel(chatModel)
            .contentRetriever(contentRetriever) // Otomatik RAG bağlamı sağlar!
            .build();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mimari Kural: RAG yanıtlarının sonuna 'Kaynaklar' (Citations) eklenmezse, kullanıcı modelin doğru mu söylediğini yoksa uydurdu mu olduğunu teyit edemez. LangChain4j `Result\<String>` kullanarak kaynak doküman ID'lerini istemciye iletin.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Traceability vulnerability: Deploying enterprise RAG without source citations makes auditability impossible. Wrap responses in `Result\<T>` or propagate segment metadata back to the client UI to establish verifiable proof chains.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ContentRetriever, veri getirme mantığını modelden tamamen bağımsızlaştıran temel SPI'dır.
- RetrievalAugmentor; sorgu dönüştürme, çoklu arama ve prompt enjeksiyonunu modülerleştirir.
- Kurumsal RAG'de kaynak gösterimi (Citation) yasal ve operasyonel bir zorunluluktur.

🎯 Key Takeaways for the Interview

- ContentRetriever abstracts information fetching into an independent, pluggable SPI.
- RetrievalAugmentor modularizes query expansion, multi-retrieval, and context injection.
- Explicit source citations are mandatory for enterprise compliance and auditability.